Лабораторная работа №4

Продвинутое использование git.

Цель работы

Получение навыков правильной работы с репозиториями git.

Теоретическое введение

Рабочий процесс Gitflow

Рабочий процесс Gitflow Workflow. Будем описывать его с использованием пакета git-flow.

Общая информация

Gitflow Workflow опубликована и популяризована Винсентом Дриссеном.
Gitflow Workflow предполагает выстраивание строгой модели ветвления с учётом выпуска проекта.
Данная модель отлично подходит для организации рабочего процесса на основе релизов.
Работа по модели Gitflow включает создание отдельной ветки для исправлений ошибок в рабочей среде.
Последовательность действий при работе по модели Gitflow:

Из ветки master создаётся ветка develop.
Из ветки develop создаётся ветка release.
Из ветки develop создаются ветки feature.
Когда работа над веткой feature завершена, она сливается с веткой develop.
Когда работа над веткой релиза release завершена, она сливается в ветки develop и master.
Если в master обнаружена проблема, из master создаётся ветка hotfix.
Когда работа над веткой исправления hotfix завершена, она сливается в ветки develop и master.

Процесс работы с Gitflow

Основные ветки (master) и ветки разработки (develop)

Для фиксации истории проекта в рамках этого процесса вместо одной ветки master используются две ветки. В ветке master хранится

официальная история релиза, а ветка develop предназначена для объединения всех функций. Кроме того, для удобства рекомендуется присваивать всем коммитам в ветке master номер версии.

При использовании библиотеки расширений git-flow нужно инициализировать структуру в существующем репозитории:

```
git flow init
```

Для github параметр Version tag prefix следует установить в v.

После этого проверьте, на какой ветке Вы находитесь:

```
git branch
```

Функциональные ветки (feature)

Под каждую новую функцию должна быть отведена собственная ветка, которую можно отправлять в центральный репозиторий для создания резервной копии или совместной работы команды. Ветки feature создаются не на основе master, а на основе develop. Когда работа над функцией завершается, соответствующая ветка сливается обратно с веткой develop. Функции не следует отправлять напрямую в ветку master.

Как правило, ветки feature создаются на основе последней ветки develop.

Создание функциональной ветки

Создадим новую функциональную ветку:

```
git flow feature start feature_branch
```

Далее работаем как обычно.

Окончание работы с функциональной веткой

По завершении работы над функцией следует объединить ветку feature_branch с develop:

```
git flow feature finish feature_branch
```

Ветки выпуска (release)

Когда в ветке develop оказывается достаточно функций для выпуска, из ветки develop создаётся ветка release. Создание этой ветки запускает следующий цикл выпуска, и с этого момента новые функции

добавить больше нельзя — допускается лишь отладка, создание документации и решение других задач. Когда подготовка релиза завершается, ветка release сливается с master и ей присваивается номер версии. После нужно выполнить слияние с веткой develop, в которой с момента создания ветки релиза могли возникнуть изменения.

Благодаря тому, что для подготовки выпусков используется специальная ветка, одна команда может дорабатывать текущий выпуск, в то время как другая команда продолжает работу над функциями для следующего.

Создать новую ветку release можно с помощью следующей команды:

```
git flow release start 1.0.0
```

Для завершения работы на ветке release используются следующие команды:

```
git flow release finish 1.0.0
```

Ветки исправления (hotfix)

Ветки поддержки или ветки hotfix используются для быстрого внесения исправлений в рабочие релизы. Они создаются от ветки master. Это единственная ветка, которая должна быть создана непосредственно от master. Как только исправление завершено, ветку следует объединить с master и develop. Ветка master должна быть помечена обновлённым номером версии.

Наличие специальной ветки для исправления ошибок позволяет команде решать проблемы, не прерывая остальную часть рабочего процесса и не ожидая следующего цикла релиза.

Ветку hotfix можно создать с помощью следующих команд:

```
git flow hotfix start hotfix_branch
```

По завершении работы ветка hotfix объединяется с master и develop:

```
git flow hotfix finish hotfix_branch
```

Семантическое версионирование

Семантический подход в версионированию программного обеспечения.

Краткое описание семантического версионирования

Семантическое версионирование описывается в манифесте семантического версионирования.

Кратко его можно описать следующим образом:

Версия задаётся в виде кортежа МАЖОРНАЯ_ВЕРСИЯ.МИНОРНАЯ_ВЕРСИЯ.ПАТЧ.

Номер версии следует увеличивать:

МАЖОРНУЮ версию, когда сделаны обратно несовместимые изменения API.

МИНОРНУЮ версию, когда вы добавляете новую функциональность, не нарушая обратной совместимости.

ПАТЧ-версию, когда вы делаете обратно совместимые исправления.

Дополнительные обозначения для предрелизных и билд-метаданных возможны как дополнения к МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ формату.

Программное обеспечение

Для реализации семантического версионирования создано несколько программных продуктов.
При этом лучше всего использовать комплексные продукты, которые используют информацию из коммитов системы версионирования.
Коммиты должны иметь стандартизованный вид.
В семантическое версионирование применяется вместе с общепринятыми коммитами.

Пакет Conventional Changelog

Пакет Conventional Changelog является комплексным решением по управлению коммитами и генерации журнала изменений.

Содержит набор утилит, которые можно использовать по-отдельности.

Общепринятые коммиты

Использование спецификации Conventional Commits.

Описание

Спецификация Conventional Commits:

Соглашение о том, как нужно писать сообщения commit'ов.
Совместимо с SemVer. Даже вернее сказать, сильно связано с семантическим версионированием.
Регламентирует структуру и основные типы коммитов.

Структура коммита

```
<type>(<scope>): <subject>
```

<BLANK LINE>
<body>
<BLANK LINE>
<footer>

Или, по-русски:

<тип>(<область>): <описание изменения>
<пустая линия>
[необязательное тело]
<пустая линия>
[необязательный нижний колонтитул]

Заголовок является обязательным.
Любая строка сообщения о фиксации не может быть длиннее 100 символов.
Тема (subject) содержит краткое описание изменения.
Используйте повелительное наклонение в настоящем времени: «изменить» ("change" not "changed" nor "changes").
Не используйте заглавную первую букву.
Не ставьте точку в конце.
Тело (body) должно включать мотивацию к изменению и противопоставлять это предыдущему поведению.
Как и в теме, используйте повелительное наклонение в настоящем времени.
Нижний колонтитул (footer) должен содержать любую информацию о критических изменениях.
Следует использовать для указания внешних ссылок, контекста коммита или другой мета информации.
Также содержит ссылку на issue (например, на github), который закрывает эта фиксация.
Критические изменения должны начинаться со слова BREAKING CHANGE: с пробела или двух символов новой строки. Затем для этого используется остальная часть сообщения фиксации.

Типы коммитов

Базовые типы коммитов
fix: — коммит типа fix исправляет ошибку (bug) в вашем коде (он соответствует PATCH в SemVer).
feat: — коммит типа feat добавляет новую функцию (feature) в ваш код (он соответствует MINOR в SemVer).
BREAKING CHANGE: — коммит, который содержит текст BREAKING

CHANGE: в начале своего не обязательного тела сообщения (body) или в подвале (footer), добавляет изменения, нарушающие обратную совместимость вашего API (он соответствует MAJOR в SemVer). BREAKING CHANGE может быть частью коммита любого типа.

revert: — если фиксация отменяет предыдущую фиксацию. Начинается с revert:, за которым следует заголовок отменённой фиксации. В теле должно быть написано: Это отменяет фиксацию <hash> (это SHA-хэш отменяемой фиксации).

Другое: коммиты с типами, которые отличаются от fix: и feat:, также разрешены. Например, @commitlint/config-conventional (основанный на The Angular convention) рекомендует: chore:, docs:, style:, refactor:, perf:, test:, и другие.

Соглашения The Angular convention

Одно из популярных соглашений о поддержке исходных кодов — конвенция Angular (The Angular convention).

Типы коммитов The Angular convention

Конвенция Angular (The Angular convention) требует следующие типы коммитов:

build: — изменения, влияющие на систему сборки или внешние зависимости (примеры областей (scope): gulp, broccoli, npm).

ci: — изменения в файлах конфигурации и скриптах CI (примеры областей: Travis, Circle, BrowserStack, SauceLabs).

docs: — изменения только в документации.

feat: — новая функция.

fix: — исправление ошибок.

perf: — изменение кода, улучшающее производительность.

refactor: — Изменение кода, которое не исправляет ошибку и не добавляет функции (рефакторинг кода).

style: — изменения, не влияющие на смысл кода (пробелы, форматирование, отсутствие точек с запятой и т. д.).

test: — добавление недостающих тестов или исправление существующих тестов.

Области действия (scope)

Областью действия должно быть имя затронутого пакета npm (как его воспринимает человек, читающий журнал изменений, созданный из сообщений фиксации).

Есть несколько исключений из правила «использовать имя

пакета»:

packaging – используется для изменений, которые изменяют структуру пакета, например, изменения общедоступного пути.

changelog – используется для обновления примечаний к выпуску в CHANGELOG.md.

отсутствует область действия – полезно для изменений стиля, тестирования и рефакторинга, которые выполняются во всех пакетах (например, style: добавить отсутствующие точки с запятой).

Соглашения @commitlint/config-conventional

Соглашение @commitlint/config-conventional входит в пакет Conventional Changelog. В целом в этом соглашении придерживаются соглашения Angular.

Выполнение лабораторной работы

1.Gроцесс установки git-flow (AVH Edition) из репозитория GitHub и проверка версии

```
[fedora@tamatafonova ~]$ cd /tmp
[fedora@tamatafonova tmp]$ git clone --depth=1 https://github.co
git
Cloning into 'gitflow-avh'...
remote: Enumerating objects: 69, done.
remote: Counting objects: 100% (69/69), done.
remote: Compressing objects: 100% (29/29), done.
remote: Total 69 (delta 45), reused 50 (delta 40), pack-reused 0
Receiving objects: 100% (69/69), 76.75 KiB | 763.00 KiB/s, done.
Resolving deltas: 100% (45/45), done.
[fedora@tamatafonova tmp]$ cd gitlow-avh
bash: cd: gitlow-avh: No such file or directory
[fedora@tamatafonova tmp]$ cd gitflow-avh
[fedora@tamatafonova gitflow-avh]$ sudo make install
```

Установка git-flow из исходного кода

2.Создание директории git-extended внутри lab04 для отдельного репозитория.

```
[fedora@tamatafonova labs]$ cd lab04
[fedora@tamatafonova lab04]$ mkdir git-extended
[fedora@tamatafonova lab04]$ cd git-extended
```

Создание структуры для лабораторной работы

3.Инициализация репозитория, создание README.md, первый коммит и проверка ветки master.

```
[fedora@tamatafonova git-extended]$ git init
Initialized empty Git repository in /home/fedora/work/study/2025-2026
intro/labs/lab04/git-extended/.git/
[fedora@tamatafonova git-extended]$ git commit -m "first commit"
On branch master
```

Инициализация Git-репозитория и первый коммит

4.Инициализация git-flow с префиксом версий v, создание ветки develop и проверка текущих веток.

```
fedora@tamatafonova git-extended]$ git flow init

hich branch should be used for bringing forth production relea
  - master
ranch name for production releases: [master]
ranch name for "next release" development: [develop]

ow to name your supporting branch prefixes?
eature branches? [feature/] v
ugfix branches? [bugfix/]
elease branches? [release/]
otfix branches? [hotfix/]
upport branches? [support/]
ersion tag prefix? []
ooks and filters directory? [/home/fedora/work/study/2025-2026
abs/lab04/git-extended/.git/hooks]
fedora@tamatafonova git-extended]$ git branch
 develop
 master
```

Инициализация git-flow

5.Добавление удаленного репозитория и проверка подключения.

```
[fedora@tamatafonova git-extended]$ git remote add origin
usage: git remote add [<options>] <name> <url>

    -f, --fetch           fetch the remote branches
    --tags                import all tags and associated objects when fetching
                          or do not fetch any tag at all (--no-tags)
    -t, --track <branch>  branch(es) to track
    -m, --master <branch>
                          master branch
    --mirror[=(push|fetch)]
                          set up remote as a mirror to push to or fetch from

[fedora@tamatafonova git-extended]$ git@github.com:matafonovataisia3-web/git-ext
bash: git@github.com:matafonovataisia3-web/git-extended.git: No such file or dir
[fedora@tamatafonova git-extended]$ git remote add origin git@github.com:matafon
/git-extended.git
[fedora@tamatafonova git-extended]$ git remote -v
```

Подключение к удаленному репозиторию GitHub

6.Отправка веток master и develop в удаленный репозиторий.

```
[fedora@tamatafonova git-extended]$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 887 bytes | 887.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:matafonovataisia3-web/git-extended.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
[fedora@tamatafonova git-extended]$ git push -u origin develop
```

Отправка веток на GitHub

7.Установка Node.js и pnpm через npm, проверка версии pnpm.

```
[fedora@tamatafonova git-extended]$ sudo dnf install nodejs
[sudo] password for fedora:
RPM Fusion for Fedora 38 - Nonfree - NVIDIA Driver                          2.7 kB/s | 1
kB     00:00
RPM Fusion for Fedora 38 - Nonfree - NVIDIA Driver                          0.0  B/s |
B     01:02
Errors during downloading metadata for repository 'rpmfusion-nonfree-nvidia-driver':
  - Status code: 404 for https://mirror.karneval.cz/pub/linux/rpmfusion/nonfree/fedora/nvid
driver/38/aarch64/repodata/repomd.xml (IP: 89.102.0.150)
  - Curl error (28): Timeout was reached for http://mirror.epn.edu.ec/rpmfusion/nonfree/fed
nvidia-driver/38/aarch64/repodata/repomd.xml [Connection timeout after 30003 ms]
  - Curl error (28): Timeout was reached for http://mirror.epn.edu.ec/rpmfusion/nonfree/fed
nvidia-driver/38/aarch64/repodata/repomd.xml [Connection timeout after 30000 ms]
Error: Failed to download metadata for repo 'rpmfusion-nonfree-nvidia-driver': Cannot downl
repomd.xml: Cannot download repodata/repomd.xml: All mirrors were tried
Ignoring repositories: rpmfusion-nonfree-nvidia-driver
Last metadata expiration check: 2:12:22 ago on Thu 05 Mar 2026 09:24:06 AM PST.
Package nodejs-1:18.19.0-1.fc38.aarch64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[fedora@tamatafonova git-extended]$ sudo dnf install pnpm
```

Установка Node.js и pnpm

8.Настройка переменных окружения pnpm, установка глобальных пакетов commitizen и standard-changelog.

```
[fedora@tamatafonova git-extended]$ pnpm setup
Appended new lines to /home/fedora/.bashrc

Next configuration changes were made:
export PNPM_HOME="/home/fedora/.local/share/pnpm"
case ":$PATH:" in
  *":$PNPM_HOME:"*) ;;
  *) export PATH="$PNPM_HOME:$PATH" ;;
esac

To start using pnpm, run:
source /home/fedora/.bashrc
[fedora@tamatafonova git-extended]$ source ~/.bashrc
[fedora@tamatafonova git-extended]$ pnpm add -g commitizen
 WARN  2 deprecated subdependencies found: glob@7.2.3, inflight@1.0.6
Packages: +150
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++
Progress: resolved 150, reused 0, downloaded 150, added 150, done

/home/fedora/.local/share/pnpm/global/5:
+ commitizen 4.3.1

Done in 38.1s using pnpm v10.30.3
[fedora@tamatafonova git-extended]$ pnpm add -g standard-changelog
```

Настройка pnpm и установка commitizen

9.Создание файла package.json с базовой информацией о проекте.

```
[fedora@tamatafonova git-extended]$ pnpm init
Wrote to /home/fedora/work/study/2025-2026/Операционные системы/os-intro/labs/lab
d/package.json
```

Инициализация Node.js проекта

10. Редактирование package.json — добавление блока config для commitizen и добавление файла в индекс.

```
[fedora@tamatafonova git-extended]$ nano package.json
[fedora@tamatafonova git-extended]$ git add package.json
[fedora@tamatafonova git-extended]$ gi status
On branch develop
Your branch is up to date with 'origin/develop'.
```

Настройка commitizen в package.json

11.Интерактивный процесс создания коммита с типом feat: и описанием “configure commitizen and package.json”.

```
[fedora@tamatafonova git-extended]$ git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0
```

Создание коммита через commitizen

12.Отправка коммита с package.json в ветку develop на GitHub.

```
fedora@tamatafonova git-extended]$ git push
numerating objects: 4, done.
ounting objects: 100% (4/4), done.
```

Отправка изменений на GitHub

13.Создание релизной ветки, генерация CHANGELOG.md, коммит и завершение релиза с созданием тега 1.0.0.

```
[fedora@tamatafonova git-extended]$ git flow release start 1.0.0
Switched to a new branch 'release/1.0.0'

Summary of actions:
- A new branch 'release/1.0.0' was created, based on 'develop'
- You are now on branch 'release/1.0.0'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

    git flow release finish '1.0.0'

[fedora@tamatafonova git-extended]$ standard-changelog --first-release
[fedora@tamatafonova git-extended]$ git add CHANGELOG.md
[fedora@tamatafonova git-extended]$ git commit -am 'chore(site): add chan
[release/1.0.0 4b98153] chore(site): add changelog
 1 file changed, 6 insertions(+)
 create mode 100644 CHANGELOG.md
[fedora@tamatafonova git-extended]$ git flow release finish 1.0.0
Switched to branch 'master'
```

Создание первого релиза (1.0.0)

14.Отправка всех веток и тегов в удаленный репозиторий.

```
fedora@tamatafonova git-extended]$ git push --all
numerating objects: 7, done.
ounting objects: 100% (7/7), done.
elta compression using up to 4 threads
ompressing objects: 100% (5/5), done.
riting objects: 100% (5/5), 2.83 KiB | 2.83 MiB/s, done.
otal 5 (delta 0), reused 0 (delta 0), pack-reused 0
o github.com:matafonovataisia3-web/git-extended.git
  0efc69b..6a09628  develop -> develop
  4ca4353..62cef04  master -> master
fedora@tamatafonova git-extended]$ git push --tags
numerating objects: 1, done.
```

Отправка релиза на GitHub

15.Создание релиза на GitHub с описанием из файла CHANGELOG.md.

```
[fedora@tamatafonova git-extended]$ gh release create v1.0.0 -F CHANGELOG.md
https://github.com/matafonovataisia3-web/git-extended/releases/tag/v1.0.0
```

Создание релиза на GitHub через gh CLI

16.Создание feature-ветки, добавление нового файла feature.js, коммит с типом feat: и завершение feature-ветки слиянием в develop.

```
[fedora@tamatafonova git-extended]$ git flow feature start feature_branch
Switched to a new branch 'vfeature_branch'

Summary of actions:
- A new branch 'vfeature_branch' was created, based on 'develop'
- You are now on branch 'vfeature_branch'

Now, start committing on your feature. When done, use:

     git flow feature finish feature_branch

[fedora@tamatafonova git-extended]$ git add .
[fedora@tamatafonova git-extended]$ git cz
```

Разработка новой функциональности

17.Создание релиза 1.2.3, обновление версии в package.json, генерация changelog и завершение релиза.

```
fedora@tamatafonova git-extended]$ git flow release start 1.2.3
anches 'develop' and 'origin/develop' have diverged.
d local branch 'develop' is ahead of 'origin/develop'.
itched to a new branch 'release/1.2.3'

mmary of actions:
A new branch 'release/1.2.3' was created, based on 'develop'
You are now on branch 'release/1.2.3'

llow-up actions:
Bump the version number now!
Start committing last-minute fixes in preparing your release
When done, run:

   git flow release finish '1.2.3'

fedora@tamatafonova git-extended]$ nano package.json
fedora@tamatafonova git-extended]$ standard-changelog
fedora@tamatafonova git-extended]$ git add CHANGELOG.md package.json
fedora@tamatafonova git-extended]$ git commit -am 'chore(site): update changelog for
elease/1.2.3 df0f470] chore(site): update changelog for v1.2.3
```

Создание второго релиза (1.2.3)

18.Отправка всех изменений на GitHub и создание второго релиза.

```
[fedora@tamatafonova git-extended]$ git push --all
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 4 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (9/9), 3.88 KiB | 1.94 MiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To github.com:matafonovataisia3-web/git-extended.git
   6a09628..34a5590  develop -> develop
   62cef04..89b3d93  master -> master
[fedora@tamatafonova git-extended]$ git push --tags
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 167 bytes | 167.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:matafonovataisia3-web/git-extended.git
 * [new tag]         1.2.3 -> 1.2.3
```

Финальная отправка и создание релиза на GitHub

Выводы

В ходе лабораторной работы были выполнены:

`Установка и настройка git-flow`

Создание репозитория с правильной структурой веток (master/develop)

Настройка семантического версионирования через Conventional Commits

Установка и использование commitizen для стандартизированных коммитов

Генерация CHANGELOG.md с помощью standard-changelog

Создание двух релизов (1.0.0 и 1.2.3) по методологии Gitflow

Разработка новой функциональности через feature-ветки

Публикация релизов на GitHub

Список литературы

1.ТУИС РУДН “Лабораторная работа №4”